Algorithms and Data Structures

Your Name · Fall 2026

Contents

1. Introduction **2**
1.1. Comparing approaches 3

2. Sorting algorithms **3**
2.1. Heapsort 4

CHAPTER 1

Introduction

These are example notes generated from the **lambda-notes** template. They show off headings, notes, callouts, tables, code, algorithms, and cross-references.

Jump ahead to the Section 2 section, or check out an external link like the Typst website.

This is a plain **note** block — use it for asides that don't need a color.

Tip

Use `callout` with a `color` and an optional `title` to highlight tips, warnings, or definitions.

Definition

A **graph** is a pair (V, E) of vertices and edges.

1.1. Comparing approaches

Approach	Time	Space
Brute force	$O(n^2)$	$O(1)$
Divide & conquer	$O(n \log n)$	$O(n)$
Dynamic programming	$O(n)$	$O(n)$

```python
def fibonacci(n: int) -> int:
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

CHAPTER 2

Sorting algorithms

2.1. Heapsort

Algorithm 1: Heapsort

```
1  procedure HEAPSORT(A, n)
2      BUILD-MAX-HEAP(A, n)
3      for i = n DOWNTO 2 do
4          ▷ Exchange A[1] with A[i]
5          A.heap-size ← A.heap.size-1
6          MAX-HEAPIFY(A, 1)
7      end
8  end
```

Heapsort runs in $O(n \log n)$ time on a sorted array of size n.